ENTERPRISEOS

Enterprise Resource & Operations Management Platform

Full Software Requirements Specification (SRS), System Design & Implementation Blueprint

Java Spring Boot • React + TypeScript • MySQL • No AI/ML Components

Document version: 1.0 | Date: 11 November 2026 | Status: Proposed design / implementation-ready blueprint

Document control and design assumptions

Item	Decision
Product model	Multi-tenant B2B SaaS; shared application, tenant-scoped data
Core stack	Java 21 LTS, Spring Boot 3.x/compatible supported release, React, TypeScript, MySQL 8.4 LTS
Architecture	Modular monolith first; event-driven integrations within one deployable backend
Tenancy	Shared database with tenant_id and enforced tenant boundaries in v1
User interfaces	Platform Admin, Company Admin, functional portals, employee self-service
Payment handling	Invoice and manual payment reconciliation first; gateway integration optional
Payroll	Configurable calculation engine; region-specific tax/statutory logic only after legal review
AI	Explicitly excluded
Delivery	Phased release with automated tests and production-readiness gates

Contents

1. Vision and scope

2. Stakeholders, personas and roles

3. Product feature catalogue

4. Detailed functional modules (15)

5. Business workflows and state machines

6. Non-functional requirements

7. Software architecture and module boundaries

8. MySQL data model

9. API specification

10. Frontend information architecture

11. Security, privacy and tenant isolation

12. Events, jobs and notifications

13. Testing and quality gates

14. DevOps, deployment and operations

15. Development roadmap and milestones

16. Definition of done, risks and future scope

Appendix A: Repository and package layout

Appendix B: Environment variables and naming conventions

1. Vision and scope

EnterpriseOS is a web-based, modular platform for organizations to operate HR, projects, customer relationships, purchasing, assets, finance and multi-level approvals. The core product goal is one trusted source of operational data, with separation between client companies and fine-grained departmental permissions.

1.1 Goals

- Onboard many independent companies on one SaaS deployment with strict data isolation.
- Replace spreadsheet-based processes with traceable, approval-driven workflows.
- Expose secure and well-documented APIs suitable for later integrations.
- Provide auditable business transactions, searchable records, reliable reports and self-service portals.
- Demonstrate professional backend engineering, not merely CRUD screens.

1.2 Out of scope for v1

- AI chatbots, LLMs, ML predictions or automated AI decision making.
- Banking, tax filing and statutory payroll guarantees for every jurisdiction.
- Full accounting general ledger and financial consolidation across subsidiaries.
- Video meetings, complete email hosting, e-commerce storefront, native mobile apps.
- Unbounded custom scripting inside workflow definitions.

1.3 Release strategy

Release	Scope	Exit condition
MVP	Company onboarding, auth/RBAC, HR core, leave, basic tasks, approvals, audit	One tenant can operate end-to-end without manual database changes
R1	Attendance, project/sprints, CRM, notification center, reports	Cross-module workflows are reliable
R2	Inventory, procurement, finance, payroll, files, webhooks	Reconciliation and financial integrity tests pass
R3	Billing, observability, resilience, operational automation	Production readiness checklist passed

2. Stakeholders, personas and roles

Role	Capabilities / boundaries
Platform Super Admin	Tenant lifecycle, subscriptions, system status; no routine access to tenant business data
Company Owner/Admin	Organization settings, users, roles, modules, billing; tenant only
HR Admin / HR Manager	Employee directory, attendance, leave policy, payroll access by assigned permission
Finance Manager / Accountant	Invoices, expenses, budgets, payment reconciliation, authorized payroll view
Project Manager / Team Lead	Project membership, sprints, tasks, time and team reports
Sales Manager / Sales Executive	Leads, pipeline, customer records, sales activities
Inventory Manager / Procurement	Products, stock movements, suppliers, purchase orders and receipts
Employee	Self-profile, attendance, leave, assigned work, allowed documents and expenses
Auditor (read-only)	Authorized audit reports and immutable records; no

	business mutation

A single user may belong to multiple tenants via separate memberships. All requests carry a validated current tenant context. Role names are defaults; permissions are the actual source of truth.

3. Product feature catalogue

Domain	Key capabilities
Tenant & company	Signup/invite, company profile, office branches, business units, tenant switching, plan limits
Identity & access	MFA, sessions, password reset, role policies, permission matrix, invitations
HR	Employee records, organizational chart, contracts, onboarding/offboarding, documents
Attendance & leave	Shifts, holidays, check-in/out, adjustments, leave balance/accrual, manager review
Payroll	Salary structure, pay periods, calculation, deductions, payslips, locked payroll runs
Projects	Projects, milestones, backlog, sprints, boards, dependencies, time logs
CRM	Leads, contacts, accounts, opportunities, pipelines, activities, quotations
Inventory	SKUs, warehouses, stock ledger, transfers, suppliers, purchase orders, goods receipt
Finance	Expenses, reimbursement, invoices, payments, budgets, aging reports
Workflow	Approval templates, conditions, ordered stages, delegation, deadlines, escalation
Communication	Notification inbox, templates, in-app events, email dispatch, announcements
Documents	Uploads, metadata, folder permissions, versions, signed download links
Analytics	Role dashboards, filters, drill-down, scheduled exports, saved report definitions
Platform billing	Trials, subscription tiers, seat limits, usage counters, invoice records
Operations	Audit trails, export requests, retention, health, backup/restore, admin tools

4. Detailed functional modules

4.1 Multi-Company SaaS & Organization Management

Company admins create tenant profiles, branches, departments, fiscal preferences, time zone, currency and branding.

Functional requirements

- Platform admin can approve/suspend/re-enable a tenant.
- Invite users using expiring single-use tokens.
- Track active users and feature availability per subscription plan.
- Support tenant membership switching without leaking prior tenant context.
- Tenant suspension disables writes without deleting historical data.

Core entities: tenants, tenant_settings, branches, departments, tenant_memberships, tenant_invitations, feature_entitlements.

Key business rules: Tenant IDs are derived from authenticated membership, not trusted from request bodies. Every company-specific query is scoped to the tenant; cross-tenant request returns not found or forbidden.

4.2 Authentication, Authorization & Security

Central identity and tenant-specific permissions for all routes and actions.

Functional requirements

- Email/password sign-in, verified email, secure reset tokens and optional TOTP MFA.
- Refresh-token rotation; session/device listing and revocation.
- Default roles and custom roles with permission grants.
- Account lockout throttling, password policies and audit of sensitive actions.
- Support step-up authentication for sensitive workflows.

Core entities: users, credentials, sessions, refresh_tokens, roles, permissions, role_permissions, user_role_assignments, mfa_factors.

Key business rules: Platform admin never gains tenant business-data rights solely from platform role. Authorization is checked in service layer and API, not only React navigation.

4.3 Employee & Department Management

Complete employee lifecycle from onboarding to exit.

Functional requirements

- Employee identifiers, contacts, manager and department hierarchy.
- Employment types, joining/exit dates, status and reporting structure.
- Onboarding/offboarding checklist and company property assignments.
- Employee profile documents with ACL-based download.
- Org chart, employee directory filters and CSV import with validation.

Core entities: employees, employment_records, employee_documents, onboarding_tasks, employee_status_history.

Key business rules: Employment status changes are versioned and audited. Sensitive personal data requires permission-specific access.

4.4 Attendance, Shifts & Leave Management

Policy-driven time and leave lifecycle.

Functional requirements

- Shift templates, rotation assignments, holidays by branch.
- Check-in/out, breaks, late marks, overtime and attendance corrections.
- Leave types, accrual rules, carry-forward and balance ledger.
- Multi-level leave approval with conflict checks.
- Monthly attendance summaries and export.

Core entities: shifts, shift_assignments, attendance_records, attendance_adjustments, leave_types, leave_balances, leave_ledger, leave_requests, holiday_calendars.

Key business rules: Overlapping leave and double check-in prevention are transactional. Timestamps stored in UTC; display based on tenant/location timezone.

4.5 Payroll & Compensation

Reproducible, controlled pay-period calculation and approval.

Functional requirements

- Salary components, allowance/deduction rules, effective-dated salary changes.
- Pay cycle creation, preview, validation and approval.
- Salary proration using eligible attendance and leave data.
- Payslip generation and downloadable employee access.
- Finalized payroll locked; corrections through explicit adjustment run.

Core entities: salary_structures, salary_components, employee_compensation, payroll_periods, payroll_runs, payroll_entries, payroll_adjustments, payslips.

Key business rules: Never silently recalculate a finalized payroll run. Region-dependent tax rules are configurable only after domain validation.

4.6 Project, Sprint & Task Management

Team work planning and delivery tracking.

Functional requirements

- Projects, members, milestones, issue types and task custom fields.
- Kanban states, swimlanes, drag-and-drop ordering.
- Sprints, backlog, story points, sprint burndown inputs.
- Task dependencies, comments, attachments, mentions and watchers.
- Time logging, workload view and activity history.

Core entities: projects, project_members, milestones, sprints, tasks, task_dependencies, task_comments, time_entries, task_history.

Key business rules: Cross-project dependency requires authorization to both resources. A task cannot depend on itself; detect dependency cycles.

4.7 CRM

Structured client and sales pipeline lifecycle.

Functional requirements

- Accounts, contacts, leads and configurable opportunity stages.
- Lead assignment, notes, reminders, call/meeting activity records.
- Quote creation and version tracking; win/loss reasons.
- Pipeline forecasts based on manually configured probabilities.
- Convert lead to customer without duplicate account records.

Core entities: crm_accounts, crm_contacts, leads, opportunities, pipeline_stages, crm_activities, quotes.

Key business rules: Contact deduplication within a tenant uses explicit matching rules. Opportunity stage changes preserve history.

4.8 Inventory, Procurement & Assets

Control stock movements and physical assets.

Functional requirements

- Product/service catalog, SKU, units and warehouse locations.
- Supplier profiles, purchase requisition and purchase order approvals.
- Goods receipts, returns, stock adjustments and stock transfers.
- Append-only stock movement ledger; available stock computed consistently.
- Asset serials, assignment, return and maintenance histories.

Core entities: products, warehouses, stock_movements, stock_balances, suppliers, purchase_requisitions, purchase_orders, goods_receipts, assets, asset_assignments.

Key business rules: No negative inventory unless explicitly permitted by tenant policy. Every stock-changing workflow uses database transaction and idempotency key.

4.9 Finance, Invoicing & Expense Management

Operational finance functions, not a complete accounting ERP ledger.

Functional requirements

- Customer invoices, line items, due dates, tax configuration and status tracking.
- Expense categories, receipts and reimbursement approvals.
- Payment records, partial payments, refunds and reconciliation status.
- Budgets by department/project and threshold alerts.
- Aging, unpaid invoices, spend trends and export.

Core entities: invoices, invoice_lines, invoice_payments, expenses, expense_attachments, reimbursements, budgets, budget_allocations.

Key business rules: Money stored as DECIMAL with explicit currency; never binary float. Invoice numbers uniquely allocated per tenant and fiscal series.

4.10 Workflow & Approval Engine

Configurable human approval journeys across HR, purchasing and finance.

Functional requirements

- Versioned approval definitions and ordered stages.
- Approval assignment by role, manager, user or department.
- Approve, reject, request changes, delegate and escalate.
- Conditional routing based on amount, department and request type.
- SLA deadlines, reminders and full decision history.

Core entities: workflow_definitions, workflow_versions, workflow_stages, workflow_instances, approval_tasks, approval_decisions, workflow_audit.

Key business rules: Published workflow version remains fixed for existing instances. A requester cannot approve their own request when separation-of-duties policy applies.

4.11 Internal Communication & Notifications

Reliable business notifications and announcements.

Functional requirements

- In-app notification center with read/unread and preferences.
- Email templates for invitations, approval requests and receipts.
- Company announcements targeted by team or branch.
- Mentions and task/approval updates.
- Delivery retry queue, dead-letter monitoring and duplicate suppression.

Core entities: notifications, notification_preferences, notification_templates, outbox_events, notification_deliveries, announcements.

Key business rules: Email failures must not roll back core approved business transactions. Users receive only events they are allowed to see.

4.12 Document & File Management

Secure tenant-controlled documents.

Functional requirements

- Folder taxonomy, upload, download and metadata.
- Versioning, document categories, expiry and access rules.
- Antivirus scanning integration when available; size/type limits.
- Short-lived signed download URLs or authenticated streaming.
- Retention policy, archive and secure deletion request processing.

Core entities: documents, document_versions, document_permissions, document_access_logs, folders.

Key business rules: Do not store untrusted uploads in publicly executable paths. File metadata and object keys are always tenant scoped.

4.13 Reports & Dashboards

Actionable operational reporting.

Functional requirements

- Role-based KPI cards and permission-aware reports.
- Attendance trends, team workload, pipeline conversion and spend.
- Date filters, tenant timezone support and saved views.
- CSV/XLSX/PDF export for permitted reports.
- Scheduled exports with file expiration and access checks.

Core entities: report_definitions, saved_report_views, report_runs, export_jobs.

Key business rules: Report queries must apply row-level tenant and permission restrictions. Large exports run asynchronously, with row-count and resource limits.

4.14 Subscription, Billing & Tenant Administration

SaaS entitlement enforcement and tenant lifecycle.

Functional requirements

- Plan catalogue, trials, seats and module entitlements.
- Subscription renewal/expiry states and usage checks.
- Invoice records and manual payment confirmation v1.

- Account owner billing page and plan-change history.
- Grace periods and safe read-only behavior on suspension.

Core entities: plans, plan_features, subscriptions, subscription_changes, billing_invoices, usage_counters.

Key business rules: Platform billing data is isolated from tenant operational finance. Never rely on frontend UI to enforce seats or feature limits.

4.15 System Administration, Audit & Monitoring

Traceability, maintainability and safe production operations.

Functional requirements

- Structured logs, correlation IDs and health endpoints.
- Append-only domain audit records with before/after references.
- Admin diagnostics, background job overview and integration delivery logs.
- Backup policy, restore drills and retention schedules.
- Export/delete request handling with legal-hold exceptions.

Core entities: audit_events, integration_deliveries, background_jobs, data_export_requests, deletion_requests.

Key business rules: Never log passwords, tokens, payment secrets or sensitive document bodies. Audit events include actor, tenant, action, resource, timestamp and result.

5. Business workflows and state machines

Employee onboarding

1. HR creates draft employee
2. Manager/department assigned
3. Invitation issued
4. Employee verifies account and completes permitted profile
5. HR verifies documents and activates employment

States: DRAFT -> INVITED -> PENDING_VERIFICATION -> ACTIVE -> OFFBOARDED

Leave approval

6. Employee creates leave request
7. Balance/overlap validations
8. Manager and optional HR approve
9. Ledger debit applied once
10. Notification sent and audit entry appended

States: DRAFT -> SUBMITTED -> IN_REVIEW -> APPROVED | REJECTED | CANCELLED

Expense reimbursement

11. Employee files expense and uploads proof
12. Rules validate limits and mandatory fields
13. Manager approval
14. Finance approval
15. Payment queued and reconciled

States: DRAFT -> SUBMITTED -> APPROVED -> READY_FOR_PAYMENT -> PAID

Purchase to stock

16. Requisition created
17. Approvals based on amount
18. Purchase order sent
19. Goods received and inspected
20. Stock movement committed atomically

States: REQUESTED -> APPROVED -> ORDERED -> PARTIALLY_RECEIVED -> RECEIVED -> CLOSED

Payroll run

21. Freeze attendance cutoff
22. Calculate draft payroll
23. Finance validates totals
24. Authorized approver finalizes
25. Payslips generated and payments recorded

States: DRAFT -> CALCULATED -> REVIEWED -> FINALIZED -> PAID

Invoice collection

26. Create invoice
27. Issue and lock numbering
28. Record payment (partial/full)
29. Apply reconciliation
30. Close when balance reaches zero

States: DRAFT -> ISSUED -> PARTIALLY_PAID -> PAID | VOID

Implementation rule: enforce transitions server-side with optimistic locking/version fields where users may modify the same entity concurrently. Business operations spanning multiple rows must commit in a single transaction; asynchronous side-effects use outbox events.

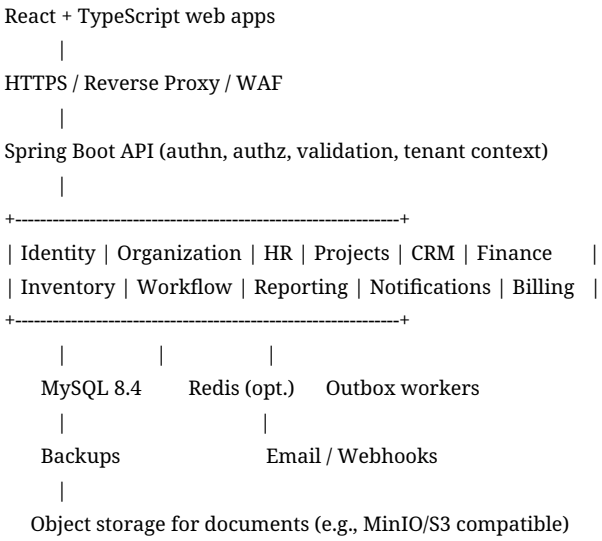
6. Non-functional requirements

Category	Initial acceptance target / requirement
Availability	Target 99.5% measured monthly after production stabilization; aspirational not guaranteed by architecture alone
API performance	P95 < 500 ms for typical indexed CRUD endpoints under documented test load; reporting queries excluded
Concurrency	Load-test an initial 100 concurrent active users; capacity plan from measurements
Security	OWASP ASVS-aligned controls; threat model and vulnerability review
Privacy	Data minimization, role restrictions, retention and tenant deletion procedures
Audit	All high-risk mutations captured with identity, timestamp, tenant, entity and outcome
Reliability	Idempotent retries for external callbacks, payment records and stock operations
Backup	Daily full backups plus binary log strategy; initial RPO 24h and RTO 8h subject to tested infrastructure
Accessibility	Keyboard navigation, focus states, semantic forms; aim for WCAG 2.2 AA

Localization	UTC storage, configurable timezone/date format, currency and language-ready UI
Maintainability	Modular packages, OpenAPI contract, migrations, tests and ADRs

7. Software architecture and module boundaries

Start with a modular monolith: a single Spring Boot deployable artifact with explicit bounded modules. Cross-module calls go through documented service interfaces and domain events, not unrestricted repository access. MySQL is the primary transactional store. React is a separate application. Redis, object storage and queue infrastructure are optional additions introduced when justified.



7.1 Backend design conventions

- Layers: controller -> application service -> domain rules -> repository.
- DTOs are versioned API contracts; never expose JPA entities directly.
- Use Bean Validation and standardized error responses (RFC 9457 Problem Details).
- Flyway/Liquibase migration per release; no production schema auto-update.
- Liquibase/Flyway migration and code deploy must have a rollback/roll-forward strategy.
- Avoid globally enabling lazy-loading across controller boundaries; use explicit projections.
- Cross-module consistency uses a transactional outbox where asynchronous delivery is necessary.

7.2 High-risk engineering decisions

Decision	Recommended v1 design
Tenancy	Shared MySQL schema with tenant_id on every tenant-owned table; enforced query specification/repository rules, composite FKs where practical
Primary IDs	UUID/ULID public IDs; internal BIGINT permitted with stable unique public IDs
Data migrations	Versioned SQL migrations and integration tests
Async events	MySQL outbox + worker; RabbitMQ added only when throughput needs justify it
Authorization	Permission-based service guards and resource ownership checks
Search	Indexed MySQL search first; external search engine only if

	necessary
File storage	Metadata in MySQL, bytes in private object store
Audit	Append-only application audit store; separate restricted DB account and retention controls

8. MySQL data model

8.1 Common columns and indexing

All tenant-owned business tables include id, tenant_id, created_at, updated_at, created_by, updated_by, version, and optional deleted_at. Global tables (users, platform plans) are clearly separated. Add composite indexes beginning with tenant_id for common tenant-scoped queries, e.g., (tenant_id, status, created_at), and unique constraints on (tenant_id, employee_code).

8.2 Entity relationship summary

users 1---N tenant_memberships N---1 tenants
| |
+---N roles +---N departments
+---N employees
employees 1---N leave_requests ---N approval_tasks
employees 1---N attendance_records
projects 1---N tasks 1---N task_comments
crm_accounts 1---N opportunities
purchase_orders 1---N goods_receipts 1---N stock_movements
invoices 1---N invoice_lines and 1---N invoice_payments
workflow_definitions 1---N workflow_versions 1---N workflow_instances

8.3 Representative table dictionary

Table	Representative fields	Constraints / indexes
tenants	id, slug, name, status, timezone, base_currency	slug unique globally
users	id, email, password_hash, verification_status	email globally unique for shared identity
tenant_memberships	tenant_id, user_id, status, joined_at	UNIQUE(tenant_id,user_id)
roles	id, tenant_id, name, system_role	UNIQUE(tenant_id,name)
permissions	id, code, module	code globally unique
employees	id, tenant_id, user_id?, employee_code, manager_id, department_id	UNIQUE(tenant_id,employee_code)
attendance_records	id, tenant_id, employee_id, work_date, in_utc, out_utc	index tenant+employee+work_date
leave_requests	id, tenant_id, employee_id, type_id, start_date, end_date, status	index tenant+status+start_date
payroll_runs	id, tenant_id, period_id, status, finalized_at	unique tenant+period+run_number
projects	id, tenant_id, code, name, status, owner_id	UNIQUE(tenant_id,code)
tasks	id, tenant_id, project_id, parent_id?, assignee_id, status, version	index tenant+project+status
crm_accounts	id, tenant_id, name, owner_id	tenant+name search index
opportunities	id, tenant_id, account_id, stage_id, amount, currency	DECIMAL for amount

products	id, tenant_id, sku, name, unit	UNIQUE(tenant_id,sku)
stock_movements	id, tenant_id, product_id, warehouse_id, delta, source_ref, idempotency_key	unique tenant+idempotency_key
invoices	id, tenant_id, invoice_no, customer_id, status, currency	UNIQUE(tenant_id,invoice_no)
expenses	id, tenant_id, employee_id, amount, currency, status	index tenant+status
workflow_instances	id, tenant_id, definition_version_id, entity_type, entity_id, status	immutable referenced version
approval_tasks	id, tenant_id, instance_id, assigned_to, state	index tenant+assignee+state
audit_events	id, tenant_id?, actor_id, action, entity, entity_id, occurred_at	append-only; partition by period if needed

8.4 Sample MySQL DDL

```
CREATE TABLE tenants (
  id BINARY(16) PRIMARY KEY,
  slug VARCHAR(80) NOT NULL UNIQUE,
  name VARCHAR(180) NOT NULL,
  status VARCHAR(24) NOT NULL,
  timezone VARCHAR(64) NOT NULL DEFAULT 'UTC',
  created_at DATETIME(6) NOT NULL
);
CREATE TABLE departments (
  id BINARY(16) PRIMARY KEY,
  tenant_id BINARY(16) NOT NULL,
  name VARCHAR(120) NOT NULL,
  parent_department_id BINARY(16) NULL,
  created_at DATETIME(6) NOT NULL,
  UNIQUE KEY uq_department (tenant_id, name),
  UNIQUE KEY uq_department_tenant_id (tenant_id, id),
  FOREIGN KEY (tenant_id) REFERENCES tenants(id),
  FOREIGN KEY (tenant_id, parent_department_id)
    REFERENCES departments(tenant_id, id)
);
```

Note: This DDL is illustrative, not the full migration set. InnoDB must be used; use migration tooling, carefully handle BINARY(16) UUID mapping, and decide on composite foreign keys for tenant-owned relationships.

9. API specification

Base path: /api/v1. All endpoints use HTTPS, validated auth context, consistent pagination, authorization, request validation and tenant scoping. Routes below are the initial contract inventory; complete OpenAPI schemas will be authored module by module.

Identity & access

```
POST /auth/login
POST /auth/refresh
POST /auth/logout
POST /auth/password/forgot
POST /auth/password/reset
POST /auth/mfa/verify
```

GET /me
GET /me/sessions
DELETE /me/sessions/{id}

Organization

POST /platform/tenants
GET /tenants/current
PATCH /tenants/current
GET /departments
POST /departments
POST /members/invitations
GET /roles
POST /roles
PUT /roles/{id}/permissions

HR

GET /employees
POST /employees
GET /employees/{id}
PATCH /employees/{id}
POST /employees/{id}/offboard
GET /employees/{id}/history

Attendance/leave

POST /attendance/check-in
POST /attendance/check-out
GET /attendance
POST /attendance/adjustments
POST /leaves/requests
GET /leaves/balances
POST /leaves/requests/{id}/cancel

Payroll

POST /payroll/runs
POST /payroll/runs/{id}/calculate
POST /payroll/runs/{id}/finalize
GET /payroll/runs/{id}/entries
GET /payroll/payslips/me

Projects

GET /projects
POST /projects
POST /projects/{id}/members
POST /projects/{id}/sprints
GET /tasks
POST /tasks
PATCH /tasks/{id}
POST /tasks/{id}/comments
POST /tasks/{id}/time-entries

CRM

GET /crm/leads
POST /crm/leads
POST /crm/leads/{id}/convert
GET /crm/accounts
POST /crm/opportunities
PATCH /crm/opportunities/{id}/stage
POST /crm/quotes

Inventory

GET /inventory/products
POST /inventory/products
GET /inventory/stock
POST /purchasing/requisitions
POST /purchasing/orders
POST /purchasing/orders/{id}/receipts
POST /inventory/transfers

Finance

GET /finance/invoices
POST /finance/invoices
POST /finance/invoices/{id}/issue
POST /finance/invoices/{id}/payments
POST /finance/expenses
POST /finance/expenses/{id}/submit
GET /finance/budgets

Workflow

GET /workflows/definitions
POST /workflows/definitions
POST /workflows/definitions/{id}/publish
GET /approvals/inbox
POST /approvals/{id}/approve
POST /approvals/{id}/reject
GET /workflows/instances/{id}/history

Common services

GET /notifications
PATCH /notifications/{id}/read
POST /documents/uploads/initiate
POST /documents/uploads/complete
GET /documents/{id}/download
POST /reports/exports
GET /audit/events

Platform billing

GET /platform/plans
POST /platform/plans
GET /billing/subscription
POST /billing/subscription/changes

GET /billing/invoices

9.1 API conventions

- Pagination: limit/cursor for high-volume feeds; page/size may be used for admin tables.
- Sorting and filters are allow-listed, never interpolated raw into SQL.
- Errors use stable codes such as TENANT_ACCESS_DENIED and VALIDATION_FAILED.
- Sensitive mutations require Idempotency-Key header where repeated submissions can double-spend or duplicate stock.
- Optimistic concurrency uses version/ETag and HTTP 409 on conflicting update.
- Webhooks use HMAC signatures, event IDs, timestamps and bounded retries.

10. Frontend information architecture

Workspace	Pages / primary screens
Public	Marketing landing, pricing, register tenant, login, password reset, verify email
Platform Admin	Tenant directory, tenant detail, billing plans, subscription exceptions, operational health
Company Admin	Overview, members/invites, roles, branches, departments, settings, feature entitlements
Employee self-service	My profile, my attendance, leave balance/requests, payslips, tasks, expenses, notifications
HR	Employee list/detail, onboarding, offboarding, shifts, policies, approvals, attendance exports
Projects	Portfolio, project board, backlog, sprint planning, timeline, task detail, time tracking
Sales	Lead list, accounts/contacts, opportunities board, quotes, activities, pipeline dashboard
Inventory	Product catalog, warehouse stock, purchase orders, receipts, transfers, asset registry
Finance	Invoice list/detail, payments, expenses, reimbursement, budgets, reports
System	Audit explorer, document center, integration logs, preferences, activity center

10.1 Frontend technical standards

- React Router with nested layouts; TanStack Query for remote server state; React Hook Form + Zod for form validation.
- Shared design system: tokens, accessible form controls, tables, dialogs, toasts, badges and loading states.
- Permissions control visibility, but backend remains source of truth for authorization.
- Error boundaries, standardized empty/error/loading states, responsive desktop-first shell.
- Do not place secrets or private business authorization rules in browser code.
- Feature folders own routes, components, validators, hooks and API clients.

11. Security, privacy and tenant isolation

11.1 Threat model and controls

Threat	Control / test
Cross-tenant IDOR	Resolve resource through tenant-scoped repository; negative

	tests with swapped IDs
Broken role access	Deny-by-default policies, object access checks, permission matrix tests
JWT theft	Short-lived access tokens, protected refresh rotation, revocation, HttpOnly secure cookies if used
CSRF / XSS	Correct CSRF strategy for cookie auth, Content Security Policy, output escaping
SQL injection	Prepared statements, parameterized ORM queries, allow-listed dynamic fields
File upload abuse	Extension/MIME validation, size caps, private storage, malware scanning pathway
Workflow abuse	Separation of duties, state transition guards, immutable decision history
Replay / duplicate requests	Idempotency keys, transaction isolation and unique constraints
Sensitive log leakage	Redaction middleware and restricted observability access
Brute force	Rate limits, account protection, MFA and anomaly alerts

11.2 Tenant isolation blueprint

31. Authenticate user and load active tenant membership from server-trusted context.
32. Resolve permission policy for module/action/resource.
33. Scope every repository query to authenticated tenant; never use caller-supplied tenant_id as authority.
34. Write tenant_id automatically via trusted service context; validate all referenced rows belong to tenant.
35. Apply composite unique keys and database constraints to prevent accidental cross-tenant associations.
36. Run cross-tenant integration tests against every endpoint and report/export path.

11.3 Data governance

- Define data retention by category (payroll, audit, files, logs) and jurisdiction before launch.
- Support access/export/deletion requests while retaining legally required finance and audit records.
- Encrypt disks/backups and use TLS; consider application-level encryption for highly sensitive fields.
- Restrict production data access; use synthetic fixtures in development and demos.

12. Events, jobs and notifications

Event / job	Trigger	Handler and guarantee
EmployeeInvited	HR creates invitation	Send email with expiring token; retry delivery
LeaveApproved	Final approval transition	Apply balance ledger once; notify employee
ExpenseApproved	Finance approval	Create reimbursement task; notify requester
GoodsReceived	Receipt finalized	Insert stock movement atomically; publish event after commit
InvoiceIssued	Invoice locks numbering	Generate PDF; email recipient; retry independently
PayrollFinalized	Authorized finalization	Produce payslips from frozen snapshot
TaskAssigned	Assignee changed	In-app/email notification based on preference
ReportExportRequested	User requests large report	Async export, expire artifact, authorized download

Use transactional outbox rows inserted in the same database transaction as business data. A polling worker dispatches unprocessed events with bounded retries and dead-letter status; consumer handlers are idempotent. This permits a reliable v1 without requiring RabbitMQ at the start.

13. Testing and quality gates

Level	Tools / scenarios	Release criterion
Unit	JUnit 5, Mockito; validators, state machines, salary rules, permission predicates	Critical business rules covered and deterministic
Integration	Spring Boot Test + Testcontainers MySQL; repositories/transactions	Migrations work on clean DB; rollback and constraints validated
Security	Tenant swap tests, privilege escalation tests, auth/session tests	Zero known high-severity authorization defects
API contract	OpenAPI validation; schema and error response tests	Published contract matches implementation
Frontend	Vitest + React Testing Library; forms and permission views	Critical UI flows pass
End-to-end	Playwright; onboarding -> leave -> approval -> payroll; invoice/payment	Core journeys pass on staging
Performance	k6/JMeter; mixed reads/writes, import/report workloads	Target latency and error budgets tested
Operational	Backup restore drill, failed job replay, deploy rollback	Documented recovery tested

13.1 Example acceptance criteria

- Given a user of Tenant A, when requesting employee ID owned by Tenant B, then access is denied and no data leaks.
- Given a leave request already approved, when approval callback is retried, then balance is deducted exactly once.
- Given two users editing the same task version, then stale update receives a conflict response.
- Given duplicate goods receipt submission using same idempotency key, then exactly one stock movement is posted.
- Given finalized payroll, then payroll numbers cannot be changed without an audited adjustment.
- Given report export request without permission, then no export file is generated or exposed.

14. DevOps, deployment and operations

14.1 Deployment profile

Recommended first production setup: Docker Compose on a properly secured VPS for learning and demo, running reverse proxy, Spring Boot API, React static build, MySQL and background worker. Managed database/object storage is preferable for critical production deployments. For higher availability, move to load-balanced instances and managed infrastructure after load/availability requirements are proven.

14.2 CI/CD pipeline

37. Pull request -> formatting, static analysis, dependency and secret checks.
38. Run backend unit/integration tests and frontend tests.
39. Build immutable Docker images with version tags and SBOM.

40. Deploy to staging and run migrations with compatibility checks.
41. Run Playwright smoke tests and tenant isolation regression.
42. Manual approval for production, backup confirmation and deploy.
43. Observe error rates, latency, job backlog and rollback on failure.

14.3 Observability and operations

- Spring Boot Actuator health, liveness and readiness probes.
- Micrometer metrics with Prometheus and Grafana (optional initial operational stack).
- Structured JSON logs with trace/request IDs and PII redaction.
- Alerts for 5xx spikes, slow SQL, low disk, backup failures and outbox backlog.
- MySQL slow query review, indexes, backups and recovery exercises.
- Runbooks for incident response, tenant suspension, restore and credential rotation.

15. Development roadmap and milestones

Phase	Illustrative solo effort	Deliverables
0. Product discovery & design	1-2 weeks	Module boundaries, SRS, screens, ERD, architecture decisions, backlog
1. Foundation	3-4 weeks	Monorepo, MySQL migrations, identity, tenant membership, roles, CI
2. Organization + HR MVP	3-4 weeks	Tenant admin, departments, employee lifecycle, documents
3. Leave + workflows	3-4 weeks	Leave balances, request state machine, approvals, notifications
4. Projects + CRM	4-6 weeks	Projects/sprints/tasks, customers/leads/opportunities
5. Attendance + payroll	4-6 weeks	Shifts, attendance, payroll calculation, safe finalization
6. Inventory + finance	5-7 weeks	Purchase/stock ledger, expenses, invoices/payment records
7. Reporting + SaaS billing	3-4 weeks	Dashboards, exports, plan entitlements, tenant billing
8. Hardening & release	3-5 weeks	Security audits, performance tests, backup/restore, deployment

Time ranges are planning estimates, not promises. Full scope is substantial; prioritize a focused 10-14 week MVP with auth, tenants, HR, leave, approvals and basic tasks. Additional modules follow once acceptance tests are stable.

15.1 MVP build order

44. Create product backlog and permission matrix.
45. Model tenants, users, memberships, roles, departments and employees.
46. Implement invitation -> login -> tenant selection -> authorization journey.
47. Add CRUD with audit and tenant isolation integration tests.
48. Implement leave policies/ledger and generic approval workflow.
49. Ship employee/admin dashboards and notifications.
50. Automate build/test/deploy and document demo scenario.

16. Definition of done, risks and future scope

16.1 Definition of done

- API and UI are implemented against explicit acceptance criteria.
- User and tenant authorization tests cover positive and negative cases.
- Migration is repeatable from clean database and on upgraded schema.
- Relevant audit records, meaningful errors and operational metrics exist.
- Tests pass in CI; OpenAPI and module documentation are updated.
- Accessibility, responsiveness and security review completed.

16.2 Risks and mitigations

Risk	Mitigation
Over-scoping the first release	Ship vertical slices and explicitly defer non-MVP modules
Accidental tenant data leakage	Central tenant context, scoped repositories, composite keys, negative automated tests
Payroll compliance mistakes	Label as configurable prototype; validate by local payroll/legal expert before real use
Financial data corruption	DECIMAL, transactions, immutable ledgers, reconciliation and idempotency
Queue or email failures	Transactional outbox, retries, dead-letter queue, observable jobs
Complex microservice operations	Start modular monolith; split only with measured evidence
Low-cost hosting resource limits	Disable optional services, benchmark, size infrastructure and monitor
Unbounded customization	Versioned safe rule schemas; no arbitrary user code execution

16.3 Future extensions (not v1 commitments)

- Mobile employee app, SSO via enterprise IdPs, localized tax modules and bank integrations.
- Custom report builder, approval workflow visual designer and customer support ticketing.
- Separate tenant database option for regulated/high-value customers.
- Event-driven service extraction, dedicated search index and data warehouse analytics.

Appendix A: Repository and package layout

```
enterpriseos/
 apps/
  web/                # React + TypeScript application
 services/
  api/                # Spring Boot modular monolith
   src/main/java/com/enterpriseos/
    common/           # errors, tenant context, pagination, security
    identity/         # users, roles, sessions
    organization/     # tenant, branches, departments
    hr/               # employees, attendance, leave, payroll
    projects/         # tasks, sprints
    crm/              # leads, accounts, opportunities
    inventory/        # stock, procurement, assets
    finance/          # invoices, expenses
```

```
    workflow/          # approval engine
    communication/     # notifications
    documents/         # files and ACLs
    reporting/         # exports, dashboards
    billing/           # plans, subscriptions
    audit/             # audit events
  src/main/resources/db/migration/
infra/
  compose/             # local compose files
  nginx/               # reverse proxy
  monitoring/          # metrics/alerts
docs/
  architecture/        # ADRs, sequence diagrams, ERD
  api/                 # OpenAPI snapshots
  runbooks/            # backup, restore, on-call docs
.github/workflows/     # CI / release pipelines
```

Appendix B: Environment variables and naming conventions

Variable	Purpose
SPRING_DATASOURCE_URL	MySQL JDBC connection
SPRING_DATASOURCE_USERNAME	Least-privilege DB user
SPRING_DATASOURCE_PASSWORD	Secret-store-managed password
APP_PUBLIC_BASE_URL	Frontend absolute URL
APP_AUTH_ISSUER	Token issuer / security context
APP_JWT_SIGNING_KEY_REF	Reference to managed signing key
APP_OBJECT_STORAGE_ENDPOINT	Private document storage
APP_MAIL_PROVIDER	SMTP or provider configuration
APP_OUTBOX_POLL_INTERVAL	Outbox worker poll frequency
APP_DEFAULT_TIMEZONE	Fallback timezone; UTC recommended

Naming conventions: Java packages lowercase by domain; DTOs request/response suffix; SQL snake_case; REST resources plural nouns; permission codes MODULE:ACTION; domain events in past tense; database migration files versioned. All example env variables are placeholders: never commit secrets.

End of specification. Implementation begins with a validated MVP backlog, ERD and access-control matrix before coding domain modules.